



## **INFORME TÉCNICO DEL PROYECTO INTEGRADOR**

**Carrera:** Tecnicatura en Programación

**Materia:** Programación II

**Título del Proyecto:** Sistema de Gestión de Pedidos Food Store

**Alumno:** Ojeda - Tarditi - Gomez - Varela

**Profesor:** Ariel Enferrel

**Fecha de entrega:** 5/6 - 22/6

# ÍNDICE

|                                                       |            |
|-------------------------------------------------------|------------|
| Introducción .....                                    | pág. 3     |
| Marco Teórico .....                                   | pág. 3/4   |
| Decisiones Técnicas y Arquitectura .....              | pág. 5/6   |
| Organización del Proyecto .....                       | pág. 6     |
| Funcionamiento del Sistema .....                      | pág. 7/8   |
| Dificultades Encontradas y Soluciones Aplicadas ..... | pág. 9     |
| Conclusiones .....                                    | pág. 9     |
| Bibliografía y Webgrafía .....                        | pág. 10    |
| Repositorio y Video Explicativo .....                 | pág. 10/11 |

# INTRODUCCIÓN

El presente proyecto consiste en un sistema de gestión denominado "Food Store", desarrollado en Java utilizando NetBeans y Maven. El objetivo principal es administrar categorías, productos, usuarios y pedidos mediante una arquitectura organizada en capas, permitiendo aplicar los conceptos estudiados en Programación II relacionados con programación orientada a objetos, excepciones, enumeraciones, interfaces, persistencia de datos y acceso a bases de datos.

El sistema permite realizar operaciones de alta, baja, modificación y consulta de las entidades involucradas, proporcionando una estructura escalable y de fácil mantenimiento.

## MARCO TEÓRICO

### Java

Java es un lenguaje de programación orientado a objetos que permite desarrollar aplicaciones robustas, portables y reutilizables. Su filosofía "Write Once, Run Anywhere" posibilita ejecutar el mismo programa en diferentes plataformas.

### Programación Orientada a Objetos

Durante el desarrollo del sistema se aplicaron los principios fundamentales de la POO:

- Encapsulamiento.
- Herencia.
- Polimorfismo.
- Abstracción.

## **Maven**

Maven es una herramienta para la administración y automatización de proyectos Java. Se utilizó para gestionar las dependencias externas y la compilación del proyecto.

## **Bases de Datos Relacionales**

Se empleó una base de datos relacional MySQL para almacenar la información de manera persistente. Este modelo organiza los datos en tablas relacionadas entre sí y permite realizar consultas mediante SQL.

## **JDBC**

Java Database Connectivity (JDBC) es una API que permite la comunicación entre las aplicaciones Java y la base de datos.

## **Manejo de Excepciones**

Se implementaron excepciones personalizadas para controlar situaciones específicas del sistema, como:

- EntidadNoEncontradaException.
- MailDuplicadoExcepcion.
- StockInvalidoException.

## **Enumeraciones**

Se utilizaron tipos enumerados para representar valores constantes, tales como:

- Estado.
- FormaPago.
- Rol.

# DECISIONES TÉCNICAS Y ARQUITECTURA

Se optó por una arquitectura en capas con el objetivo de separar responsabilidades y facilitar el mantenimiento del código.

La estructura implementada se compone de:

## **Capa de Entidades**

Contiene las clases del dominio del problema:

- Categoria
- Producto
- Usuario
- Pedido
- DetallePedido
- Base

Estas clases representan los objetos principales del sistema.

## **Capa DAO (Data Access Object)**

Encargada del acceso a la base de datos:

- CategoriaDAO
- ProductoDAO
- UsuarioDAO
- PedidoDAO

Su función es ejecutar las operaciones CRUD sobre las tablas correspondientes.

## **Capa de Servicios**

Contiene la lógica de negocio:

- CategoriaService
- ProductoService
- UsuarioService
- PedidoService

Esta capa actúa como intermediaria entre la interfaz y la persistencia.

### **Capa de Configuración**

La clase ConexionDB centraliza la conexión con la base de datos MySQL.

### **Capa de Excepciones**

Implementa excepciones personalizadas para un mejor control de errores.

### **Clase Main**

La clase Main proporciona un menú interactivo mediante consola para gestionar todas las funcionalidades del sistema.

## **ORGANIZACIÓN DEL PROYECTO**

El proyecto se encuentra dividido en los siguientes paquetes:

### **config**

Responsable de la conexión con la base de datos.

### **dao**

Implementa el acceso y manipulación de datos persistentes.

### **entities**

Contiene las clases del modelo del sistema.

### **services**

Implementa la lógica de negocio.

### **exception**

Contiene las excepciones personalizadas.

### **enums**

Define los valores constantes del sistema.

### **main**

Contiene la clase principal encargada de ejecutar la aplicación.

# FUNCIONAMIENTO DEL SISTEMA

El menú principal presenta las siguientes opciones:

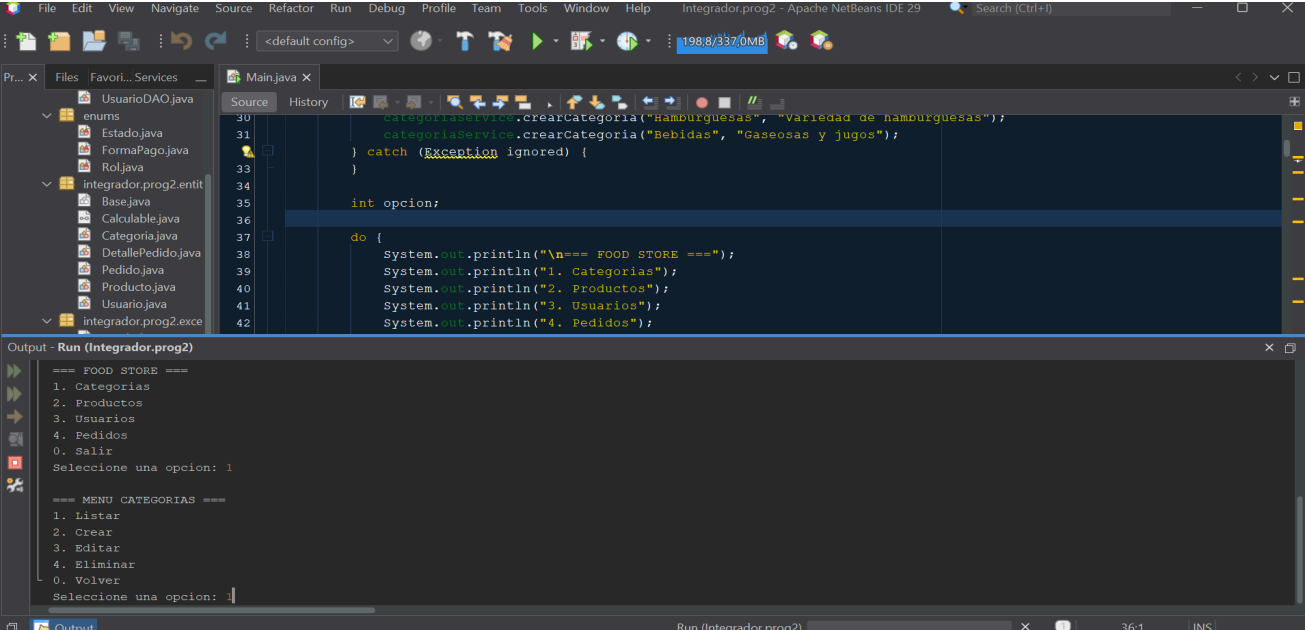
1. Gestión de categorías.
2. Gestión de productos.
3. Gestión de usuarios.
4. Gestión de pedidos.

Cada módulo permite realizar operaciones de:

- Alta.
- Baja.
- Modificación.
- Consulta.

Además, el sistema valida la existencia de entidades y controla errores mediante excepciones específicas.

## CAPTURAS DEL FUNCIONAMIENTO DEL SISTEMA



The screenshot displays the Apache NetBeans IDE interface. The top toolbar shows various development tools. The left sidebar contains a project explorer with the following structure:

- enums
  - UsuarioDAO.java
- integrador.prog2.entit
  - Base.java
  - Calculable.java
  - Categoria.java
  - DetallePedido.java
  - Pedido.java
  - Producto.java
  - Usuario.java
- integrador.prog2.exe

The main editor window shows the source code of `Main.java`. The code includes a `try-catch` block for creating categories and a `do-while` loop for the main menu. The output window at the bottom shows the program's execution:

```
==== FOOD STORE ====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opcion: 1

==== MENU CATEGORIAS ====
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione una opcion: 1
```

```
--- LISTADO DE PEDIDOS ---
ID Pedido: 1 | Fecha: 2026-06-20
Usuario: Troxi | Estado: PENDIENTE | Pago: EFECTIVO
TOTAL: $200000.0
Detalles: (Sin detalles)
-----
ID Pedido: 2 | Fecha: 2026-06-20
Usuario: Troxi | Estado: PENDIENTE | Pago: TARJETA
TOTAL: $400000.0
Detalles: (Sin detalles)
-----
```

|   | ID | Categoría    | Descripción              | Estado |
|---|----|--------------|--------------------------|--------|
| ▶ | 1  | Hamburguesas | Variedad de hamburguesas | Activa |
|   | 2  | Bebidas      | Gaseosas y jugos         | Activa |

|   | ID | Producto             | Categoría    | Precio Unitario | Unidades en Stock | Disponible para Venta | Estado |
|---|----|----------------------|--------------|-----------------|-------------------|-----------------------|--------|
| ▶ | 1  | Hamburguesa especial | Hamburguesas | \$2000          | 26                | Sí                    | Activo |

|   | Nro Pedido | Categoría    | Producto a Preparar  | Cantidad | Detalle/Especificación del Plato |
|---|------------|--------------|----------------------|----------|----------------------------------|
| ▶ | 1          | Hamburguesas | Hamburguesa especial | 4        | Con bacon, cheddar y cebolla     |

|   | Nro Pedido | Cliente     | Producto Comprado    | Cantidad | Precio Unitario | Subtotal Renglón | Total del Pedido | Estado    |
|---|------------|-------------|----------------------|----------|-----------------|------------------|------------------|-----------|
| ▶ | 1          | Troxi Ojeda | Hamburguesa especial | 4        | \$2000          | \$8000           | \$8000           | PENDIENTE |

# DIFICULTADES ENCONTRADAS Y SOLUCIONES APLICADAS

## Conexión con la base de datos

Uno de los principales inconvenientes fue establecer correctamente la conexión con MySQL. Este problema se solucionó incorporando la dependencia mysql-connector-j mediante Maven y centralizando la conexión en la clase ConexionDB.

## Manejo de excepciones

Durante las pruebas surgieron errores relacionados con búsquedas de entidades inexistentes y validaciones de stock. Para resolverlos se implementaron excepciones personalizadas.

## Organización del proyecto

La cantidad creciente de clases dificultó inicialmente el mantenimiento del código. Por ello se adoptó una arquitectura en capas, separando entidades, servicios y acceso a datos.

# CONCLUSIONES

El desarrollo del sistema permitió aplicar los conocimientos adquiridos en Programación II, especialmente los conceptos de programación orientada a objetos, manejo de excepciones, enumeraciones, interfaces y persistencia de datos mediante MySQL.

Durante el proceso de desarrollo también se presentaron diferentes dificultades relacionadas con la conexión a la base de datos, la validación de información y la organización del código. Sin embargo, cada uno de estos inconvenientes fue resuelto mediante la investigación, las pruebas realizadas y la aplicación de las herramientas estudiadas en la materia.

En conclusión, el proyecto representó una oportunidad para afianzar los conocimientos teóricos aprendidos en clase y transformarlos en una

aplicación funcional, organizada y capaz de seguir ampliándose con nuevas características en el futuro.

## **BIBLIOGRAFÍA Y WEBGRAFÍA**

Oracle. Java Documentation.

<https://docs.oracle.com/en/java/>

Apache Maven Project.

<https://maven.apache.org/>

MySQL by Ariel Enferrel

<https://www.sql-practice.com/>

NetBeans IDE.

<https://netbeans.apache.org/>

Charly Cimino.

[Home - Charly Cimino](#)

---

## **REPOSITORIO Y VIDEO EXPLICATIVO**

Repositorio GitHub:

<https://github.com/FrancoThomas3/Integrador.prog2UTN.git>

Video explicativo:

 Video Integrador Prog2.mov